CADERNO DE ROTEIROS PRÁTICOS

Unidade Curricular: Programação para Internet 1

CST Sistemas para Internet – Semestre 1

Sumário

1 Visão Geral da Unidade Curricular **5**

2 Roteiro Prático 01 — Fundamentos da Web, HTML5 e Versionamento **6**
2.1 Introdução . . . 6
2.2 Parte 1 — Fundamentos da Internet . . . 7
2.2.1 Como funciona uma página web? . . . 7
2.3 Parte 2 — Estrutura Base do Documento HTML5 . . . 7
2.3.1 Estrutura mínima obrigatória . . . 7
2.4 Parte 3 — Comandos Básicos HTML5 . . . 9
2.5 Parte 4 — Trabalhando com Links . . . 11
2.5.1 Exemplo . . . 11
2.6 Parte 5 — Versionamento com Git e GitHub . . . 12
2.7 Checklist Final de Verificação . . . 13

3 Roteiro Prático 02 — HTML5 Semântico, CSS Externo e Estrutura Profissional **14**
3.1 Introdução . . . 14
3.2 Fundamentação Teórica — HTML5 Semântico . . . 14
3.3 Principais Tags Estruturais do HTML5 . . . 15
3.4 Boas Práticas . . . 15
3.5 Parte 1 — Organização Profissional do Projeto . . . 17
3.6 Parte 2 — Estrutura HTML Semântica . . . 17
3.7 Parte 3 — Introdução ao CSS Externo . . . 19
3.8 Parte 4 — Inserindo Imagens . . . 20
3.9 Checklist Final . . . 20
3.10 Entrega da atividade . . . 21
3.11 Fundamentação Complementar — Propriedades CSS Essenciais . . . 22
3.12 Principais Propriedades CSS . . . 22
3.13 Dicas Essenciais — Git e GitHub . . . 23

4 Roteiro Prático 03 — Publicação com GitHub Pages e Aprimoramento Visual **24**
4.1 Parte 1 — Ajustes Estruturais . . . 24
4.2 Parte 2 — Aprimoramento Visual com CSS . . . 24
4.3 Parte 3 — Publicação com GitHub Pages . . . 27
4.3.1 Passo 1 — Atualizar repositório . . . 27
4.3.2 Passo 2 — Ativar GitHub Pages . . . 27
4.3.3 Parte 4 — Verificação da Publicação . . . 27
4.4 Checklist Final . . . 28

5 Roteiro Prático 04 — Evoluindo o Layout com Flexbox **31**
5.1 Objetivos da Aula . . . 31
5.2 Introdução . . . 31
5.3 Preparação do Ambiente . . . 31
5.3.1 Clonar o Repositório . . . 31
5.4 Estado Inicial do Projeto . . . 31
5.5 Parte 1 — Melhorando o Menu com Flexbox . . . 32
5.6 Parte 2 — Inserindo os Cards (HTML) . . . 32
5.6.1 Atualize o `index.html` . . . 32
5.7 Parte 3 — Organizando os Cards com Flexbox . . . 33
5.8 Parte 4 — Estilizando os Cards . . . 34
5.8.1 Efeito de interação . . . 34
5.9 Parte 5 — Melhorando o Menu . . . 34
5.10 Resultado Final Esperado . . . 35
5.11 Versionamento . . . 35
5.12 Checklist Final . . . 35
5.13 Repositório Oficial da Unidade Curricular . . . 36

6 Roteiro Prático 05 — Responsividade e Evolução dos Cards **37**
6.1 Objetivos da Aula . . . 37
6.2 Introdução . . . 37
6.3 Preparação do Ambiente . . . 37
6.3.1 Situação 1 — Projeto já existe no computador . . . 37
6.3.2 Situação 2 — Novo computador ou projeto não encontrado . . . 37
6.3.3 Situação 3 — Dúvidas sobre o estado do projeto . . . 38
6.4 Parte 1 — Adicionando Imagens . . . 39
6.4.1 Atualize o HTML . . . 39
6.5 Parte 2 — Ajustando os Cards (Substituição) . . . 39
6.5.1 Substitua a classe `.cards` . . . 39
6.5.2 Substitua a classe `.card` . . . 39

6.6 Parte 3 — Ajustando o Menu . . . 40
6.7 Parte 4 — Adicionando Novos Estilos . . . 40
6.7.1 Imagens nos cards . . . 40
6.7.2 Efeito visual . . . 40
6.8 Parte 5 — Responsividade . . . 41
6.9 Resultado Final Esperado . . . 41
6.10 Versionamento . . . 42
6.11 Checklist Final . . . 42
6.12 Desafio Extra . . . 42
6.13 Resumo das Propriedades CSS Utilizadas . . . 42
6.14 Repositório Oficial da Unidade Curricular . . . 44

7 Roteiro Prático 06 — Formulários e Estilização Avançada com HTML5 e CSS3 45
7.1 Objetivos da Aula . . . 45
7.2 Introdução . . . 45
7.3 Parte 1 — Estrutura do Formulário . . . 45
7.3.1 Atualize o HTML . . . 45
7.4 Parte 2 — Estrutura Visual do Formulário . . . 46
7.5 Parte 3 — Estilização dos Campos . . . 47
7.6 Parte 4 — Interação com o Usuário . . . 47
7.7 Parte 5 — Botão Interativo . . . 47
7.8 Parte 6 — Validação Visual . . . 48
7.9 Parte 7 — Responsividade . . . 48
7.10 Tabela de Referência — HTML5 e CSS Utilizados . . . 48
7.11 Desafio Extra . . . 49
7.12 Repositório Oficial do Roteiro 6 . . . 50

8 Roteiro Prático 07 — Manipulação da Árvore DOM 51
8.1 O que é o DOM? . . . 51
8.2 Sintaxe Moderna: A Metáfora da Seta (Arrow Functions) . . . 51
8.3 Parte 1 — Selecionando e Alterando Elementos . . . 51
8.4 Parte 2 — Tabelas de Referência de Métodos . . . 52
8.5 Parte 3 — Criação Dinâmica de Elementos . . . 52
8.6 Parte 4 — Praticando com Estados e Atributos . . . 53
8.7 Laboratório de Mini-Jogos Lúdicos . . . 55

9 Roteiro Prático 08 — Requisições Assíncronas e Fetch API 58
9.1 Entendendo o JSON . . . 58
9.2 A Metáfora da Esteira de Produção (Fetch Chain) . . . 58

9.3 Exploração Prática — O Projeto ViaCEP 58
9.4 Parte Final — Mantendo a Organização 59

1 Visão Geral da Unidade Curricular

Esta unidade curricular foca no desenvolvimento de interfaces interativas e dinâmicas utilizando as tecnologias fundamentais da web. O cronograma está organizado em 8 roteiros progressivos:

1. **Fundamentos da Web e Estrutura HTML5**
2. **HTML5 Semântico e Estrutura de Mini-Sites**
3. **Estilização com CSS3 e Box Model**
4. **Layouts Modernos: Flexbox e Grid**
5. **Responsividade e Frameworks CSS**
6. **Lógica de Programação com JavaScript**
7. **Roteiro Intermediário: Manipulação da Árvore DOM**
   Foco intensivo em métodos de seleção, criação e eventos.
8. **Consumo de APIs e Requisições Assíncronas (Fetch)**
   Integração de dados externos (JSON) no frontend.

2 Roteiro Prático 01 — Fundamentos da Web, HTML5 e Versionamento

Objetivos

Objetivos da Aula

Ao final desta aula o estudante será capaz de:

- Compreender a evolução da Internet e o papel do HTML;
- Identificar os principais componentes de um documento HTML5;
- Criar uma página web estruturada corretamente;
- Utilizar tags básicas para organização de conteúdo;
- Executar versionamento com Git e GitHub desde o início do projeto.

Introdução

A Internet transformou profundamente a forma como pessoas, empresas e governos produzem, compartilham e consomem informação.

Sua origem remonta à década de 1960, com a ARPANET, projeto militar norte-americano que buscava criar uma rede descentralizada de comunicação entre computadores.

Na década de 1990, Tim Berners-Lee desenvolveu a World Wide Web (WWW), criando:

- O protocolo HTTP;
- O conceito de URL;
- A linguagem HTML;
- O primeiro navegador web.

A Web evoluiu em diferentes fases:

- **Web 1.0**: páginas estáticas;
- **Web 2.0**: interação e redes sociais;
- **Web 3.0**: dados inteligentes e descentralização.

O HTML (HyperText Markup Language) é a base estrutural de praticamente toda aplicação web.

Mesmo sistemas complexos desenvolvidos em outras linguagens entregam HTML ao navegador.

Parte 1 — Fundamentos da Internet

2.2.1 Como funciona uma página web?

Quando um usuário digita uma URL no navegador:

1. O navegador envia uma requisição HTTP;
2. O servidor responde enviando um documento HTML;
3. O navegador interpreta o HTML;
4. O conteúdo é renderizado na tela.

Atividade Prática 1 - Pesquisa dirigida

Pesquise e escreva uma definição objetiva (máx. 4 linhas cada) para:

- ARPANET
- Tim Berners-Lee
- W3C
- HTTP
- URL
- Navegador Mosaic
- Web 1.0, 2.0 e 3.0

Em seguida, responda:
Qual foi o principal impacto da Web na transformação da sociedade digital?

Parte 2 — Estrutura Base do Documento HTML5

2.3.1 Estrutura mínima obrigatória

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>Prog. Internet I - 2026</title>
</head>
```

```
<body>

</body>
</html>
```

Atividade prática 2 — Criando o Projeto

- Criar diretório `prog-internet-2026`;
- Criar arquivo `index.html`;
- Inserir a estrutura base HTML5;
- Testar no navegador.

Micro-Commit 1 — Estrutura Inicial

```
git init
git add .
git commit -m "Estrutura inicial HTML5"
```

Parte 3 — Comandos Básicos HTML5

TAG	Definição
<!DOCTYPE html>	Define documento HTML5
<html>	Estrutura geral da página
<head>	Informações da página
<meta charset="UTF-8">	Codificação
<title>	Título da aba
<body>	Conteúdo visível
<h1> até <h6>	Títulos
<p>	Parágrafo
<hr>	Linha horizontal
<ul> / <li>	Lista não ordenada
<pre>	Texto pré-formatado
<a>	Hiperlink

Atividade 3 — Inserindo Conteúdo

Adicionar ao `<body>`:

- Título da disciplina;
- Subtítulo com seu nome;
- Parágrafo explicando o que você entende por Internet;
- Lista com 3 tecnologias que deseja aprender;
- Duas linhas horizontais (`<hr>`);
- Bloco `<pre>` contendo:
  - Seu curso;
  - Seu semestre;
  - Data atual.

Após implementar, teste novamente no navegador.

Micro-Commit 2 — Conteúdo Estruturado

Registre a evolução do projeto:

```
git add .
git commit -m "Adicionando conteúdo estruturado"
```

Parte 4 — Trabalhando com Links

2.5.1 Exemplo

```
<a href="https://www.ifsc.edu.br" target="_blank">
    Página do IFSC
</a>
```

Atividade 4 — Trabalhando com Links e Comentários

Adicionar ao final da página:

- Um link para o site do IFSC;
- Um link para o site do W3C;
- Um comentário HTML explicando qual foi sua maior dificuldade na atividade.

Os links devem abrir em nova aba utilizando o atributo `target="_blank"`.
Após implementar, teste novamente no navegador.

Micro-Commit 3 — Links e Comentários

Registre a evolução do projeto:

```
git add .
git commit -m "Adicionando links externos e comentário HTML"
```

Parte 5 — Versionamento com Git e GitHub

Atividade 5 — Publicando o Projeto no GitHub

Passo 1 — Criar repositório no GitHub

- Acesse: `https://github.com`
- Clique em **New Repository**
- Nomeie como: `prog-internet-2026`
- Selecione como **Public**
- NÃO marque a opção para criar README automaticamente
- Clique em **Create repository**

Passo 2 — Executar comandos no terminal

No diretório do seu projeto, execute:

```
echo "# prog-internet-2026" >> README.md
git init
git add README.md
git add .
git commit -m "Primeiro commit - Estrutura inicial"
git branch -M main
git remote add origin https://github.com/SEU-USUARIO/prog-internet-2026.git
git push -u origin main
```

Se solicitado, utilize seu usuário e token de autenticação do GitHub.

Resultado Esperado

- O arquivo `index.html` deve estar visível no repositório;
- O arquivo `README.md` deve estar criado;
- O repositório deve conter pelo menos um commit inicial.

Boas Práticas a partir desta Aula

- Todo código desenvolvido será versionado;
- Cada etapa significativa deverá gerar um commit;
- O histórico de commits fará parte da avaliação formativa;
- Organização e clareza no repositório são critérios avaliativos.

Checklist Final de Verificação

Checklist Técnico

Antes de finalizar o roteiro, verifique:

- O documento inicia com `<!DOCTYPE html>`;
- A tag `<html>` contém o atributo `lang="pt-BR"`;
- O `<meta charset="UTF-8">` está corretamente definido;
- O código está indentado e organizado;
- Os links utilizam `target="_blank"`;
- O repositório contém pelo menos três commits distintos;
- O arquivo está corretamente publicado no GitHub.

Observação Técnica Importante

A tag `<font>` foi descontinuada no HTML5.
Toda formatação visual deve ser realizada utilizando CSS, que será introduzido nos próximos roteiros.

3 Roteiro Prático 02 — HTML5 Semântico, CSS Externo e Estrutura Profissional

Objetivos

Objetivos da Aula

Ao final desta aula o estudante será capaz de:

- Utilizar corretamente HTML5 semântico;
- Organizar múltiplas páginas interligadas;
- Criar e conectar um arquivo CSS externo;
- Trabalhar com imagens utilizando caminhos relativos;
- Estruturar diretórios de forma profissional;
- Aplicar versionamento incremental com Git.

Introdução

No roteiro anterior construímos uma página HTML básica e iniciamos o versionamento com Git.

Agora avançaremos para um modelo mais próximo de aplicações reais, organizando um mini-site com múltiplas páginas, estrutura semântica adequada e separação entre conteúdo (HTML) e estilo (CSS).

A separação entre estrutura e apresentação é um dos pilares do desenvolvimento web moderno. O HTML é responsável pela organização do conteúdo, enquanto o CSS define a aparência visual da aplicação.

Neste roteiro construiremos um mini-site institucional com organização profissional.

Fundamentação Teórica — HTML5 Semântico

O HTML5 introduziu uma abordagem mais semântica para organização de páginas web. Isso significa que as tags passaram a descrever o significado estrutural do conteúdo.

A semântica melhora:

- Organização do código;
- Acessibilidade;
- SEO (Search Engine Optimization);
- Manutenção e escalabilidade do projeto.

Principais Tags Estruturais do HTML5

Tag	Função Estrutural
<header>	Cabeçalho da página ou seção.
<nav>	Área de navegação principal.
<main>	Conteúdo principal da página.
<section>	Agrupamento temático de conteúdo.
<article>	Conteúdo independente (post, card, notícia).
<aside>	Conteúdo complementar ou lateral.
<footer>	Rodapé da página ou seção.
<link>	Conexão com arquivos externos (CSS).
<img>	Inserção de imagem (com atributo alt obrigatório).

Boas Práticas

Evite estruturar páginas apenas com . Utilize as tags semânticas para tornar o código mais claro e profissional.

Dicas Úteis

Atalhos Importantes no VS Code para HTML

- Digite `!` e pressione `TAB` para gerar automaticamente a estrutura básica do HTML5.
- Para formatar automaticamente o código:
  - `OPTION + SHIFT + I` [Mac].
  - `ALT + SHIFT + I` [Linux].
  - `ALT + SHIFT + F` [Windows].
- Utilize `CTRL + /` para comentar ou descomentar linhas.
- Use `CTRL + Espaço` para ativar sugestões automáticas.
- Utilize `CTRL + S` frequentemente para salvar alterações.

Dica Profissional: Acostume-se a trabalhar com código bem indentado e organizado desde o início. Isso faz parte da avaliação formativa.

Dicas Úteis

Sequência Recomendada de Uso do Git

Sempre que finalizar uma etapa do trabalho, utilize a seguinte sequência:

```
git status
git add .
git commit -m "Descrição clara da alteração"
git push
```

Boa prática:

- Faça commits pequenos e frequentes;
- Utilize mensagens descritivas;
- Evite acumular muitas alterações em um único commit.

Parte 1 — Organização Profissional do Projeto

Estrutura recomendada:

```
prog-internet-2026/

  roteiro02/
      index.html
      sobre.html
      contato.html
      assets/
          css/
              style.css
          img/
              perfil.jpg
```

Atividade 1 — Criando Estrutura Base

- Criar pasta `roteiro02`;
- Criar os três arquivos HTML;
- Criar pasta `assets/css`;
- Criar pasta `assets/img`;
- Criar arquivo `style.css`.

Micro-Commit 1

```
git add .
git commit -m "Criando estrutura profissional roteiro 02"
```

Parte 2 — Estrutura HTML Semântica

No arquivo `index.html`:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <title>Mini-Site</title>
```

```
    <link rel="stylesheet" href="assets/css/style.css">
</head>
<body>

<header>
    <h1>Meu Mini-Site</h1>
</header>

<nav>
    <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="sobre.html">Sobre</a></li>
        <li><a href="contato.html">Contato</a></li>
    </ul>
</nav>

<main>
    <section>
        <h2>Conteúdo Principal</h2>
        <p>Texto da seção.</p>
    </section>
</main>

<footer>
    <p>Seu nome - 2026</p>
</footer>

</body>
</html>
```

Atividade 2 — Implementando Estrutura

- Implementar estrutura nas três páginas;
- Garantir navegação funcional;
- Testar todos os links.

Micro-Commit 2

```
git add .
git commit -m "Implementando estrutura semantica completa"
```

Parte 3 — Introdução ao CSS Externo

No arquivo `style.css`:

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
}

header {
    background-color: #003366;
    color: white;
    padding: 15px;
}

nav ul {
    list-style: none;
}

nav li {
    display: inline;
    margin-right: 15px;
}

footer {
    margin-top: 20px;
    font-size: 14px;
}
```

Atividade 3 — Aplicando CSS

- Criar e conectar arquivo CSS;
- Verificar funcionamento nas três páginas;

- Ajustar estilos conforme necessário.

Micro-Commit 3

```
git add .
git commit -m "Adicionando estilo CSS externo"
```

Parte 4 — Inserindo Imagens

No arquivo `index.html`:

```
<img src="assets/img/perfil.jpg" alt="Foto de perfil">
```

Atividade 4 — Trabalhando com Imagem

- Inserir imagem na página inicial;
- Utilizar atributo alt;
- Garantir caminho relativo correto.

Micro-Commit 4

```
git add .
git commit -m "Adicionando imagem ao mini-site"
```

Checklist Final

Verificação Estrutural

- Estrutura de diretórios organizada;
- HTML semântico implementado corretamente;
- CSS externo funcionando;
- Navegação funcional;
- Imagem carregando corretamente;
- Pelo menos 4 commits realizados.

Publicação Final no GitHub

Após revisar o checklist:

- Verifique se todos os arquivos estão salvos;
- Execute a sequência completa de versionamento;
- Certifique-se de que o repositório remoto está atualizado.

```
git status
git add .
git commit -m "Finalizacao roteiro 02"
git push
```

Resultado esperado:
Todos os arquivos do roteiro02 devem estar visíveis no GitHub.

Entrega da atividade

Entrega da Atividade

Para concluir esta atividade, o estudante deverá:

- Confirmar que o repositório está atualizado no GitHub;
- Copiar o link público do repositório;
- Enviar apenas o link na tarefa correspondente no SIGAA.

Exemplo de link esperado:
`https://github.com/seu-usuario/prog-internet-2026`
Importante:

- O repositório deve estar público;
- O histórico de commits será considerado na avaliação;
- Não será necessário enviar arquivos compactados (.zip).

Fundamentação Complementar — Propriedades CSS Essenciais

O CSS (Cascading Style Sheets) é responsável pela apresentação visual da página. Ele permite definir cores, espaçamentos, fontes, posicionamento e organização visual.

Principais Propriedades CSS

Propriedade	Função
color	Define a cor do texto.
background-color	Define a cor de fundo do elemento.
font-family	Define a família da fonte.
font-size	Define o tamanho da fonte.
margin	Define o espaçamento externo do elemento.
padding	Define o espaçamento interno do elemento.
border	Define bordas ao redor do elemento.
width / height	Define largura e altura do elemento.
display	Define o tipo de exibição (block, inline, inline-block).
text-align	Define alinhamento do texto.

Exemplo Prático

```
section {
    background-color: #f4f4f4;
    padding: 20px;
    margin: 10px;
}
```

Dicas Essenciais — Git e GitHub

O Git é a ferramenta de controle de versão utilizada nesta disciplina. Abaixo estão os principais comandos que você utilizará com frequência.

Comando	Função
git status	Mostra o estado atual do repositório (arquivos modificados, adicionados ou pendentes de commit).
git add .	Adiciona todos os arquivos modificados para a área de preparação (staging area).
git commit -m ”mensagem”	Cria um novo commit com uma descrição clara da alteração realizada.
git push	Envia os commits locais para o repositório remoto no GitHub.
git pull	Atualiza seu repositório local com alterações feitas no repositório remoto.

4 Roteiro Prático 03 — Publicação com GitHub Pages e Aprimoramento Visual

Objetivos

Objetivos da Aula

Ao final desta aula o estudante será capaz de:

- Refinar a estrutura visual do mini-site;
- Organizar o layout utilizando container centralizado;
- Aplicar efeitos visuais simples com CSS;
- Configurar e publicar o site utilizando GitHub Pages;
- Disponibilizar um link público funcional.

Introdução

Até este momento construímos um mini-site estruturado com HTML5 semântico e CSS externo.

Agora daremos um passo importante: tornar o projeto visualmente mais organizado e publicá-lo na internet utilizando o GitHub Pages.

A publicação de um site estático permite que qualquer pessoa acesse sua aplicação por meio de um endereço público.

Este é o primeiro contato com deploy web na disciplina.

Parte 1 — Ajustes Estruturais

Adicionar dentro do <head> de todas as páginas:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Isso prepara o site para futura responsividade.

Parte 2 — Aprimoramento Visual com CSS

Atualize o arquivo `style.css` com:

```
body {
    font-family: Arial, sans-serif;
    margin: 0;
    background-color: #f2f2f2;
}

.container {
    width: 80%;
    margin: 0 auto;
}

header {
    background-color: #003366;
    color: white;
    padding: 20px;
    text-align: center;
}

nav ul {
    list-style: none;
    padding: 0;
    text-align: center;
}

nav li {
    display: inline;
    margin: 0 15px;
}

nav a {
    text-decoration: none;
    color: #003366;
    font-weight: bold;
}

nav a:hover {
    color: #ff6600;
```

```
}

section {
    background-color: white;
    padding: 20px;
    margin-top: 20px;
    border-radius: 8px;
    box-shadow: 0px 2px 6px rgba(0,0,0,0.1);
}

footer {
    margin-top: 30px;
    padding: 15px;
    text-align: center;
    font-size: 14px;
}
```

Atividade 1 — Aplicando Container

- Envolver o conteúdo principal dentro de uma ;
- Testar o alinhamento centralizado;
- Verificar se todas as páginas mantêm consistência visual.

Micro-Commit 1

```
git add .
git commit -m "Aprimorando layout com container e efeitos visuais"
```

Parte 3 — Publicação com GitHub Pages

4.3.1 Passo 1 — Atualizar repositório

```
git status
git add .
git commit -m "Preparando publicacao no GitHub Pages"
git push
```

4.3.2 Passo 2 — Ativar GitHub Pages

No repositório no GitHub:

- Acessar **Settings**;
- Clicar em **Pages**;
- Em Source, selecionar:
  - Branch: main
  - Folder: /root
- Salvar.

O GitHub fornecerá um endereço público no formato:
`https://seu-usuario.github.io/prog-internet-2026/`

4.3.3 Parte 4 — Verificação da Publicação

Atividade 2 — Teste de Acesso Público

- Acessar o link fornecido pelo GitHub;
- Navegar entre as páginas;
- Verificar funcionamento da navegação;
- Testar em outro navegador ou dispositivo.

Checklist Final

Verificação Geral

- Layout centralizado;
- CSS aplicado corretamente;
- Efeito hover funcionando;
- Navegação funcional;
- GitHub Pages ativo;
- Site acessível por URL pública.

Entrega da Atividade

O estudante deverá enviar no SIGAA:

- Link público do GitHub Pages;
- Link do repositório no GitHub.

Não é necessário enviar arquivos compactados.

6. FAQ – Problemas Comuns com Git e GitHub

Durante as atividades práticas com Git e GitHub, alguns erros podem ocorrer, principalmente em ambientes de laboratório Linux. Abaixo estão os problemas mais comuns e como resolvê-los rapidamente.

6.1 Erro: Please tell me who you are

Mensagem típica:

```
*** Please tell me who you are.
Run
  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"
```

Causa: O Git precisa identificar quem está realizando o commit. Nome e e-mail ainda não foram configurados na máquina.

Solução:

Execute no terminal:

```
git config --global user.name "Seu Nome Completo"
git config --global user.email "seu_email@exemplo.com"
```

Exemplo:

```
git config --global user.name "Maria da Silva"
git config --global user.email "maria@email.com"
```

Após isso, tente novamente:

```
git commit -m "Mensagem do commit"
```

Observação: Se o computador for compartilhado, pode-se configurar apenas para o repositório atual:

```
git config user.name "Seu Nome"
git config user.email "seu_email@exemplo.com"
```

—

6.2 Erro: ”fatal: unable to auto-detect email address”

Causa: Mesmo problema do item anterior: o Git não tem nome e e-mail configurados.

Solução: Configurar o usuário conforme indicado na seção 6.1.

—

6.3 Erro ao dar push – senha não aceita

Mensagem típica:

```
remote: Support for password authentication was removed
fatal: Authentication failed
```

Causa: O GitHub não permite mais autenticação usando apenas senha.

Soluções possíveis:

Opção 1 – Utilizar Token de Acesso (PAT):

- Gerar um token no GitHub.
- Usar o token no lugar da senha ao executar o `git push`.

Opção 2 – Configurar autenticação via SSH:

- Gerar chave SSH com:

  ```
  ssh-keygen -t ed25519 -C "seu_email@exemplo.com"
  ```

- Adicionar a chave pública nas configurações do GitHub.

—

6.4 Verificar configurações atuais do Git

Para conferir se o nome e e-mail estão configurados:

```
git config --list
```

Ou apenas as configurações globais:

```
git config --global --list
```

—

6.5 Erro: ”nothing to commit”

Mensagem típica:

```
nothing to commit, working tree clean
```

Causa: Nenhuma alteração foi adicionada à área de stage.
Solução:
Verificar o status:

```
git status
```

Adicionar arquivos:

```
git add nome_do_arquivo
```

Ou todos os arquivos:

```
git add .
```

Depois realizar o commit novamente.
—

6.6 Dica Importante

Sempre que ocorrer um erro:

- Leia atentamente a mensagem exibida no terminal.
- Utilize o comando `git status` para entender a situação atual do repositório.
- Evite repetir comandos sem compreender o erro.

Grande parte dos problemas com Git está relacionada à configuração inicial ou à autenticação.

5 Roteiro Prático 04 — Evoluindo o Layout com Flexbox

Objetivos da Aula

Objetivos

Ao final desta aula o estudante será capaz de:

- Recuperar corretamente um projeto existente com Git;
- Evoluir um site já desenvolvido;
- Aplicar Flexbox para organização de layout;
- Criar componentes visuais (cards);
- Melhorar a organização visual da página;
- Trabalhar com evolução incremental de código.

Introdução

Nos roteiros anteriores construímos um mini-site com estrutura básica em HTML e iniciamos a estilização com CSS.

Neste roteiro, iremos evoluir o projeto existente, melhorando o layout e adicionando novos elementos visuais.

Importante: não iremos criar um novo projeto. Vamos continuar exatamente de onde paramos.

Preparação do Ambiente

5.3.1 Clonar o Repositório

```
git clone https://github.com/SEU-USUARIO/prog-internet-2026.git
cd prog-internet-2026/roteiro-3
```

Dicas Úteis

Sempre utilize o `git clone`. Evite recriar projetos manualmente.

Estado Inicial do Projeto

Arquivo: index.html (Roteiro 03)

```
<main>
    <section>
        <h2>Conteúdo Principal</h2>
        <p>Texto da seção.</p>
    </section>
</main>
```

Parte 1 — Melhorando o Menu com Flexbox

```
nav ul {
    display: flex;
    justify-content: center;
    gap: 20px;
    list-style: none;
    padding: 0;
}
```

Atividade 1

- Aplicar Flexbox ao menu;
- Centralizar os itens;
- Ajustar espaçamento.

Parte 2 — Inserindo os Cards (HTML)

Agora vamos evoluir o HTML adicionando uma nova seção.

5.6.1 Atualize o `index.html`

```
<main>
    <section>
        <h2>Conteúdo Principal</h2>
        <p>Texto da seção.</p>
    </section>

    <section class="cards">
        <div class="card">
```

```
            <h3>Projeto 1</h3>
            <p>Descrição do projeto.</p>
        </div>

        <div class="card">
            <h3>Projeto 2</h3>
            <p>Descrição do projeto.</p>
        </div>

        <div class="card">
            <h3>Projeto 3</h3>
            <p>Descrição do projeto.</p>
        </div>
    </section>
</main>
```

Atividade 2

- Adicionar a seção de cards;
- Criar ao menos 3 cards;
- Inserir títulos e descrições.

Parte 3 — Organizando os Cards com Flexbox

Agora vamos organizar os cards lado a lado.

```
.cards {
    display: flex;
    gap: 15px;
    margin-top: 20px;
}
```

Dicas Úteis

Sem o Flexbox, os elementos ficam empilhados verticalmente.

Parte 4 — Estilizando os Cards

```
.card {
    background-color: white;
    padding: 15px;
    border-radius: 8px;
    flex: 1;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}
```

5.8.1 Efeito de interação

```
.card:hover {
    transform: scale(1.03);
}
```

Atividade 3

- Aplicar estilização nos cards;
- Testar o efeito hover;
- Ajustar espaçamento.

Parte 5 — Melhorando o Menu

```
nav a {
    padding: 5px 10px;
    background-color: #ddd;
    border-radius: 5px;
}

nav a:hover {
    background-color: #bbb;
}
```

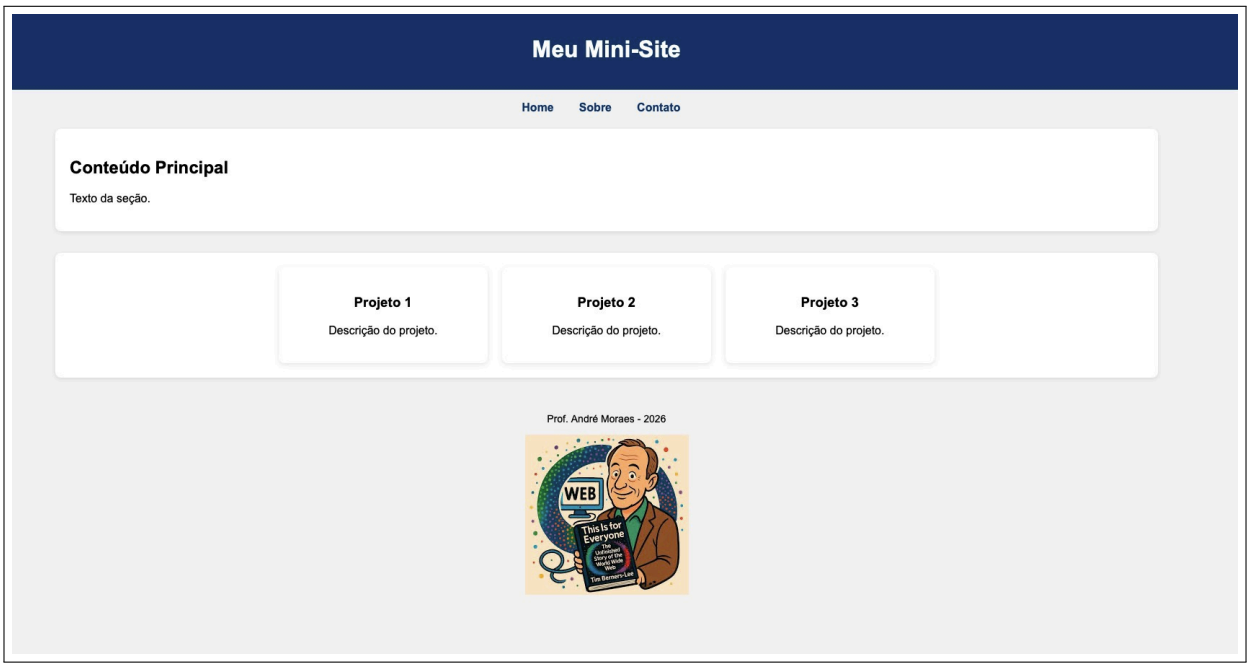

Figura 1: Layout final com cards organizados utilizando Flexbox.

Resultado Final Esperado

Versionamento

```
git add .
git commit -m "Adiciona seção de cards com flexbox"
git push
```

Checklist Final

- Projeto recuperado com Git;
- Menu com Flexbox;
- Seção de cards criada;
- Cards estilizados;
- Layout organizado;
- Alterações versionadas.

Repositório Oficial da Unidade Curricular

Repositório Oficial da Disciplina

https://github.com/chameoandre/uc-prog-internet-2026.git

6 Roteiro Prático 05 — Responsividade e Evolução dos Cards

Objetivos da Aula

Objetivos

Ao final desta aula o estudante será capaz de:

- Evoluir o layout criado com Flexbox;
- Trabalhar com imagens dentro dos cards;
- Diferenciar quando substituir ou adicionar código CSS;
- Aplicar responsividade com Media Queries;
- Melhorar a organização visual do site.

Introdução

No roteiro anterior, criamos uma seção de cards utilizando Flexbox.

Agora, vamos evoluir esse layout com melhorias visuais e adaptação para diferentes tamanhos de tela.

Nesta aula, será importante compreender a diferença entre:

- **Substituir código existente** (quando corrigimos ou melhoramos algo);
- **Adicionar código novo** (quando incluímos funcionalidades).

Preparação do Ambiente

Antes de iniciar a aula, é necessário garantir que o projeto esteja atualizado na sua máquina.

6.3.1 Situação 1 — Projeto já existe no computador

Se você já possui o projeto salvo na máquina, acesse a pasta do projeto e execute:

```
git pull
```

Isso irá atualizar o projeto com as versões mais recentes do GitHub.

6.3.2 Situação 2 — Novo computador ou projeto não encontrado

Se você está em outro computador ou não possui o projeto salvo, será necessário cloná-lo novamente:

```
git clone https://github.com/SEU-USUARIO/prog-internet-2026.git
```

Após isso, acesse a pasta do projeto:

```
cd prog-internet-2026
```

E entre na pasta do último roteiro:

```
cd roteiro-5
```

6.3.3 Situação 3 — Dúvidas sobre o estado do projeto

Caso você não tenha certeza se seu projeto está atualizado:

- Execute `git pull`;
- Verifique se os arquivos estão presentes;
- Em caso de erro, consulte o professor.

Dicas Úteis

Importante:
Nunca recrie o projeto manualmente. Sempre utilize o `git clone` para manter o histórico de versões.

Parte 1 — Adicionando Imagens

6.4.1 Atualize o HTML

```
<div class="card">
    <img src="assets/img/projeto1.jpg" alt="Projeto 1">
    <h3>Projeto 1</h3>
    <p>Descrição do projeto.</p>
</div>
```

Atividade 1

- Inserir imagens em todos os cards;
- Garantir que as imagens estejam na pasta correta;
- Testar o carregamento no navegador.

Parte 2 — Ajustando os Cards (Substituição)

Agora vamos **substituir** parte do CSS existente para melhorar o comportamento dos cards.

6.5.1 Substitua a classe `.cards`

```
.cards{
    display: flex;
    gap: 20px;
    justify-content: center;
    text-align: center;
}
```

6.5.2 Substitua a classe `.card`

```
.card{
    display: flex;
    flex-direction: column;
    background-color: white;
    padding: 20px;
    width: 250px;
    border-radius: 8px;
```

```
    box-shadow: 0px 2px 6px rgba(0,0,0,0.1);
    text-align: center;
}
```

Dicas Úteis

A propriedade `flex-direction` só funciona corretamente quando usamos `display: flex`.

Parte 3 — Ajustando o Menu

Remova o trecho abaixo do seu CSS:

```
nav li {
    display: inline;
}
```

Dicas Úteis

Quando utilizamos Flexbox, não precisamos utilizar `display: inline`.

Parte 4 — Adicionando Novos Estilos

Agora vamos **adicionar** novos estilos ao final do arquivo CSS.

6.7.1 Imagens nos cards

```
.card img {
    width: 100%;
    border-radius: 5px;
    margin-bottom: 10px;
}
```

6.7.2 Efeito visual

```
.card:hover {
    transform: translateY(-5px);
    box-shadow: 0px 4px 12px rgba(0,0,0,0.2);
    transition: all 0.3s ease;
}
```

Parte 5 — Responsividade

Agora vamos tornar o layout adaptável para telas menores.

```
@media(max-width: 768px){
    .cards{
        flex-direction: column;
        align-items: center;
    }
}
```

Atividade 2

- Redimensionar o navegador;
- Verificar comportamento dos cards;
- Testar em tela menor.

Resultado Final Esperado

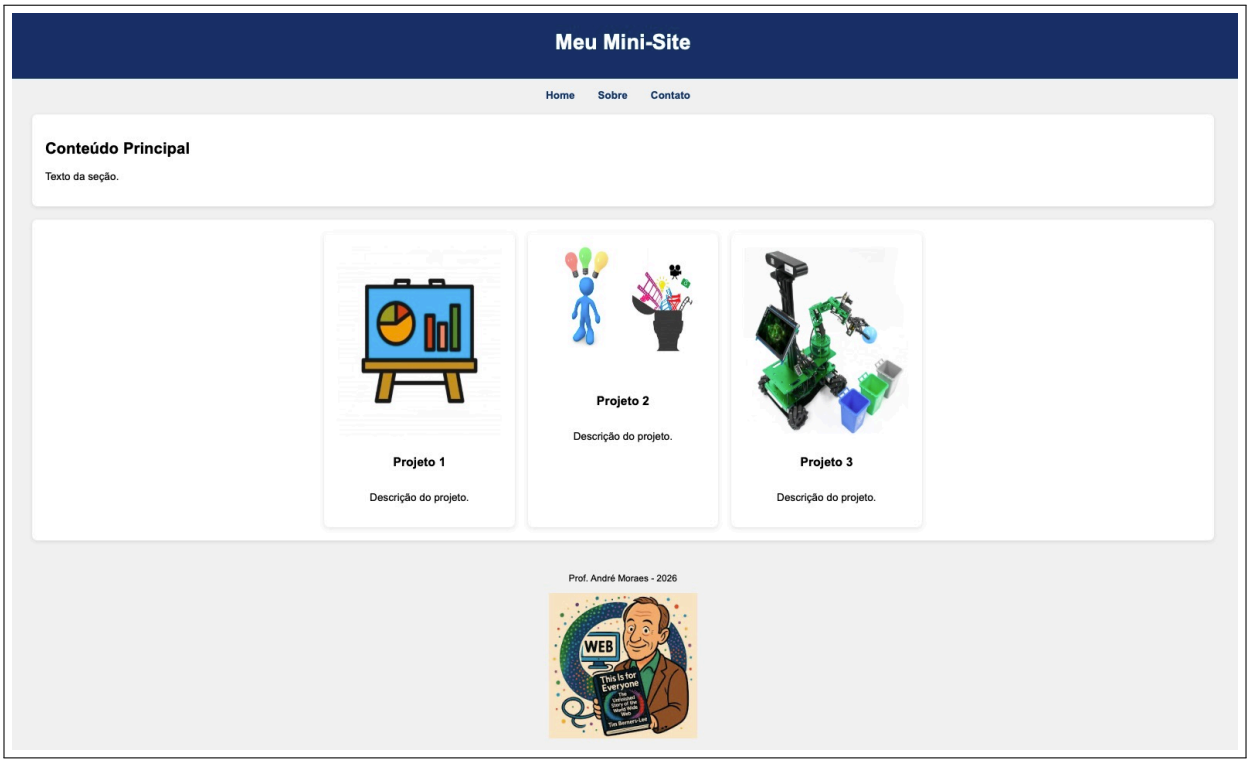

Figura 2: Cards com imagens e comportamento responsivo.

Versionamento

```
git add .
git commit -m "Melhoria visual e responsividade dos cards"
git push
```

Checklist Final

- Cards com imagens;
- CSS atualizado corretamente;
- Código substituído onde necessário;
- Novos estilos adicionados;
- Responsividade funcionando;
- Projeto versionado.

Desafio Extra

- Criar um quarto card;
- Alterar cores dos cards;
- Inserir botão nos cards;
- Personalizar layout.

Resumo das Propriedades CSS Utilizadas

A tabela abaixo apresenta as principais propriedades CSS utilizadas ao longo dos roteiros e suas funções dentro do site desenvolvido.

Propriedade	Onde foi usada	Função
display: flex	Menu e cards	Organiza elementos em linha de forma flexível
justify-content	Menu e cards	Alinha elementos horizontalmente
gap	Menu e cards	Define espaçamento entre elementos
flex-direction	Cards	Define a direção dos elementos (coluna nos cards)
width	Container e cards	Define largura dos elementos
margin	Container	Centraliza elementos na página
padding	Cards e sections	Define espaçamento interno
background-color	Header, section, cards	Define cor de fundo
color	Textos e links	Define cor do texto
border-radius	Cards e sections	Arredonda os cantos dos elementos
box-shadow	Cards e sections	Adiciona sombra para destacar elementos
text-align	Vários elementos	Alinha o texto
text-decoration	Links	Remove sublinhado dos links
font-weight	Links	Deixa o texto em negrito
hover	Links e cards	Aplica efeito visual ao passar o mouse
transform	Cards	Cria efeito de animação (movimento/escala)
@media	Layout geral	Permite responsividade para diferentes telas

Dicas Úteis

Dica de estudo:
Revise esta tabela antes das próximas aulas. Todas as propriedades aqui foram utilizadas no seu próprio projeto.

Repositório Oficial da Unidade Curricular

Repositório Oficial da Disciplina

`Configura o conteúdo do roteiro 5 online`

7 Roteiro Prático 06 — Formulários e Estilização Avançada com HTML5 e CSS3

Objetivos da Aula

Objetivos

Ao final desta aula o estudante será capaz de:

- Criar formulários modernos com HTML5;
- Aplicar validações automáticas;
- Melhorar a experiência do usuário (UX);
- Estilizar formulários com CSS3;
- Compreender o funcionamento dos principais atributos utilizados.

Introdução

Nesta aula, iremos evoluir o formulário criado anteriormente, tornando-o mais completo e visualmente profissional.

Além disso, vamos entender melhor como funcionam os atributos de validação do HTML5 e como o CSS pode melhorar a interação com o usuário.

Parte 1 — Estrutura do Formulário

Nesta etapa, vamos reorganizar o formulário, incluindo novos campos e melhorando a experiência do usuário.

7.3.1 Atualize o HTML

```
<form class="form-contato">

    <h2>Formulário de Contato</h2>

    <label for="nome">Nome:</label>
    <input type="text" id="nome" required minlength="3" placeholder="Digite seu nome">
```

```
    <label for="email">Email:</label>
    <input type="email" id="email" required placeholder="exemplo@email.com">

    <label for="telefone">Telefone:</label>
    <input type="tel" id="telefone" pattern="[0-9]{10,11}" placeholder="Somente números"

    <label for="assunto">Assunto:</label>
    <select id="assunto">
        <option value="">Selecione</option>
        <option value="duvida">Dúvida</option>
        <option value="sugestao">Sugestão</option>
        <option value="contato">Contato</option>
    </select>

    <label for="mensagem">Mensagem:</label>
    <textarea id="mensagem" rows="4" required></textarea>

    <button type="submit">Enviar</button>

</form>
```

Dicas Úteis

O HTML5 já possui validações automáticas para vários tipos de campo.

Parte 2 — Estrutura Visual do Formulário

Agora vamos estruturar o formulário visualmente.

```
form {
    display: flex;
    flex-direction: column;
    gap: 12px;
    background-color: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0px 2px 6px rgba(0,0,0,0.1);
    max-width: 500px;
    margin: 20px auto;
```

```
}
```

Parte 3 — Estilização dos Campos

Nesta etapa, melhoramos a aparência dos campos.

```
input, textarea, select {
    padding: 10px;
    border: 1px solid #ccc;
    border-radius: 5px;
    font-size: 14px;
}
```

Parte 4 — Interação com o Usuário

Aqui adicionamos feedback visual ao usuário.

```
input:focus, textarea:focus, select:focus {
    outline: none;
    border-color: #003366;
    box-shadow: 0 0 5px rgba(0,51,102,0.3);
}
```

Parte 5 — Botão Interativo

```
button {
    background-color: #003366;
    color: white;
    padding: 12px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    transition: background-color 0.3s ease;
}

button:hover {
    background-color: #0055aa;
}
```

Parte 6 — Validação Visual

Agora vamos mostrar ao usuário se o campo está correto ou não.

```
input:valid {
    border-color: green;
}

input:invalid {
    border-color: red;
}
```

Parte 7 — Responsividade

```
@media(max-width: 768px){
    form {
        width: 90%;
    }
}
```

Tabela de Referência — HTML5 e CSS Utilizados

Elemento	Exemplo	Função
required	input required	Campo obrigatório
type="email"	input type="email"	Valida formato de email
minlength	minlength="3"	Define tamanho mínimo
pattern	pattern="[0-9]10,11"	Define padrão (regex)
placeholder	placeholder="Digite..."	Texto de ajuda
display:flex	display: flex	Layout flexível
gap	gap: 10px	Espaçamento entre elementos
:focus	input:focus	Estilo ao clicar
:valid	input:valid	Campo válido
:invalid	input:invalid	Campo inválido
transition	transition: 0.3s	Animação suave
@media	@media(max-width:768px)	Responsividade

Desafio Extra

- Validar telefone com JavaScript;
- Exibir mensagem personalizada sem usar alert;
- Limpar o formulário após envio;
- Criar validação mais avançada.

Repositório Oficial do Roteiro 6

Repositório Oficial do Roteiro 6

`Configura o conteúdo do roteiro 6 online`

8 Roteiro Prático 07 — Manipulação da Árvore DOM

Objetivos

- Dominar os métodos de seleção: `getElementById` e `querySelector`;
- Manipular dinamicamente o conteúdo da página (`innerText`, `innerHTML`);
- Alterar estilos CSS e gerenciar classes (`classList`);
- Criar e remover elementos HTML via JavaScript (`createElement`, `appendChild`).

O que é o DOM?

O **DOM (Document Object Model)** é a representação em árvore dos elementos do seu HTML que o navegador cria para que o JavaScript possa interagir com eles.

Sintaxe Moderna: A Metáfora da Seta (Arrow Functions)

Dicas Úteis

O Atalho do Chef (=>)
Imagine que escrever código é como pedir um prato em um restaurante. Na forma antiga (`function`), você tinha que preencher um formulário longo: "Eu declaro que gostaria de uma função que recebe um pedido e faz tal coisa...".
Com a **Arrow Function** (=>), você usa um atalho. O símbolo => é uma seta que aponta diretamente do **evento** para a **ação**. É como dizer: "Quando o botão for clicado **=>** mude a cor". É mais rápido, limpo e moderno!

Parte 1 — Selecionando e Alterando Elementos

A Intenção do Código

Nesta primeira atividade, vamos ensinar o navegador a "olhar" para um título específico e trocar a roupa (cor) dele. O JavaScript vai localizar o ID único e aplicar o novo estilo instantaneamente.

Atividade prática 1 — Seletores e Texto

Crie um arquivo `dom-test.html` e implemente:

```
<h1 id="titulo">Olá Mundo!</h1>
<button onclick="mudarTexto()">Clique Aqui</button>

<script>
function mudarTexto() {
    const titulo = document.getElementById("titulo");
    titulo.innerText = "JavaScript funcionando!";
    titulo.style.color = "blue";
}
</script>
```

Parte 2 — Tabelas de Referência de Métodos

Guia de Métodos de Seleção

Método	Descrição
`getElementById()`	Busca um elemento pelo seu ID único.
`querySelector()`	Busca o primeiro elemento que valide o seletor CSS (ex: .classe, #id, tag).
`querySelectorAll()`	Busca uma lista de todos os elementos que validam o seletor.

Parte 3 — Criação Dinâmica de Elementos

Este é um dos conceitos mais importantes: fazer o HTML "aparecer" na tela via código.

A Intenção do Código

Vamos criar do zero novos elementos, como se estivéssemos montando peças de LEGO na tela. Em vez de escrever o HTML manualmente, o JavaScript vai fabricar uma nova `<li>` toda vez que você clicar no botão.

Atividade prática 2 — A Fábrica de Elementos

Vamos criar uma lista onde cada clique no botão adiciona um novo item:

```
<ul id="minhaLista"></ul>
<button id="add">Adicionar Item</button>

<script>
const lista = document.getElementById("minhaLista");
const botao = document.getElementById("add");

botao.addEventListener("click", () => {
    // 1. Criar o elemento
    const novoItem = document.createElement("li");

    // 2. Dar conteúdo ao elemento
    novoItem.innerText = "Novo Item na Lista";

    // 3. Adicionar na página
    lista.appendChild(novoItem);
});
</script>
```

Parte 4 — Praticando com Estados e Atributos

A Intenção do Código

Agora vamos lidar com números e imagens. No primeiro exemplo, o JavaScript vai guardar um valor na memória (uma variável) e aumentá-lo a cada clique. No segundo, vamos manipular o atributo `src` para trocar uma imagem por outra, simulando um interruptor.

Atividade prática 3 — O Contador de Cliques

Crie um contador simples:

```
<h2 id="contador">0</h2>
<button id="btnSoma">Contar +1</button>

<script>
let valor = 0;
const display = document.getElementById("contador");

document.getElementById("btnSoma").addEventListener("click", () => {
    valor++;
    display.innerText = valor;
});
</script>
```

Atividade prática 4 — Luz Acesa e Luz Apagada

Troque a imagem dinamicamente (use links de imagens de lâmpadas ou placeholders):

```
<img id="lampada" src="off.png" style="width:100px">
<button id="btnInterruptor">Ligar/Desligar</button>

<script>
const img = document.getElementById("lampada");
let acesa = false;

document.getElementById("btnInterruptor").addEventListener("click", () => {
    acesa = !acesa;
    img.src = acesa ? "on.png" : "off.png";
});
</script>
```

Laboratório de Mini-Jogos Lúdicos

A Intenção do Código

Para consolidar o aprendizado, vamos transformar o código em jogo! Nestes desafios, o JavaScript deixa de ser apenas uma ferramenta de formulário e passa a gerenciar estados de jogo, aleatoriedade e reações rápidas ao mouse.

Atividade prática 5 — Jogo de Adivinhação

Conceito: O computador escolhe um número e você tenta adivinhar. O JS compara os valores e te dá dicas.

```
<input type="number" id="palpite" placeholder="0 a 10">
<button id="btnChutar">Chutar!</button>
<p id="feedback"></p>

<script>
const segredo = Math.floor(Math.random() * 11);
const campo = document.getElementById("palpite");
const texto = document.getElementById("feedback");

document.getElementById("btnChutar").addEventListener("click", () => {
    const chute = Number(campo.value);
    if (chute === segredo) {
        texto.innerText = "Parabéns! Você acertou!";
        texto.style.color = "green";
    } else {
        texto.innerText = chute > segredo ? "Muito alto!" : "Muito baixo!";
        texto.style.color = "red";
    }
});
</script>
```

Atividade prática 6 — O Botão Fugitivo

Conceito: Um botão que foge do mouse! Vamos aprender a mudar a posição absoluta do elemento e contar quantas vezes você falhou.

```
<p>Erros: <span id="contadorErros">0</span></p>
<button id="fugitivo" style="position: absolute; left: 50%; top: 50%;">
    Pegue-me!
</button>

<script>
const btn = document.getElementById("fugitivo");
const placar = document.getElementById("contadorErros");
let erros = 0;

btn.addEventListener("mouseover", () => {
    erros++;
    placar.innerText = erros;

    // Mudar posição aleatoriamente
    const x = Math.random() * (window.innerWidth - 100);
    const y = Math.random() * (window.innerHeight - 50);

    btn.style.left = `${x}px`;
    btn.style.top = `${y}px`;
});

btn.addEventListener("click", () => alert("Você me pegou!"));
</script>
```

Atividade prática 7 — Jokenpô (Emoji Edition)

Conceito: Pedra, Papel e Tesoura. O JavaScript decide o sorteio do computador e compara com a sua escolha.

```
<button onclick="jogar('PEDRA')">PEDRA</button>
<button onclick="jogar('PAPEL')">PAPEL</button>
<button onclick="jogar('TESOURA')">TESOURA</button>
<h3 id="resultadoJogo">Escolha um!</h3>

<script>
const opcoes = ['PEDRA', 'PAPEL', 'TESOURA'];

function jogar(player) {
    const pc = opcoes[Math.floor(Math.random() * 3)];
    const res = document.getElementById("resultadoJogo");

    if (player === pc) {
        res.innerText = `Empate! Ambos escolheram ${pc}`;
    } else if (
        (player === 'PEDRA' && pc === 'TESOURA') ||
        (player === 'PAPEL' && pc === 'PEDRA') ||
        (player === 'TESOURA' && pc === 'PAPEL')
    ) {
        res.innerText = `Você Ganhou! ${player} vence ${pc}`;
    } else {
        res.innerText = `Você Perdeu! ${pc} vence ${player}`;
    }
}
</script>
```

9 Roteiro Prático 08 — Requisições Assíncronas e Fetch API

Objetivos

- Compreender o formato de dados JSON;
- Utilizar a Fetch API para buscar dados externos;
- Integrar o consumo de APIs com a manipulação do DOM.

Entendendo o JSON

JSON é o formato padrão de troca de mensagens na web moderna. Ele funciona como um objeto JavaScript formatado como texto.

A Metáfora da Esteira de Produção (Fetch Chain)

Dicas Úteis

A Esteira de Pedidos

Imagine uma esteira de produção em uma lanchonete:

1. `fetch` [O Pedido] **=>** Você faz o pedido no balcão (internet).
2. `json()` [A Cozinha] **=>** O cozinheiro recebe o pedido bruto e o transforma em um prato pronto para comer (objeto JS).
3. `dados` [O Garçom] **=>** Agora você pode pegar o prato e colocar na mesa (mostrar na tela).

O encadeamento de `.then() => ...` é o movimento da esteira passando o pedido de uma mão para a outra.

Exploração Prática — O Projeto ViaCEP

No mundo real, as páginas não são estáticas. Elas precisam buscar informações em outros servidores (APIs). Nesta etapa, vamos transformar seu formulário em uma ferramenta de consulta em tempo real.

A Intenção do Código

Nesta atividade, vamos conectar as pontas: o usuário digita um CEP no seu `index.html` ⇒ O JavaScript faz uma requisição para o servidor do ViaCEP ⇒ O servidor nos responde um pacote de dados (JSON) ⇒ Nós "abrimos" esse pacote e usamos o DOM para preencher o endereço na tela automaticamente.

Atividade prática 3 — Integração Completa (Fetch + DOM)

Dentro da sua pasta `roteiro-8/`, abra o arquivo `index.html` e implemente a lógica de busca:

```
const cep = document.getElementById("cep").value;

fetch(`https://viacep.com.br/ws/${cep}/json/`)
    .then(response => response.json())
    .then(dados => {
        if (dados.erro) {
            document.getElementById("resultado").innerText = "Erro!";
        } else {
            document.getElementById("resultado").innerText =
               `${dados.logradouro}, ${dados.localidade}`;
        }
    });
```

Tabela de Referência — Comandos de Rede

Comando	O que faz?
`fetch(url)`	Inicia a requisição para o servidor.
`.then()`	Aguarda a promessa ser resolvida para executar a ação.
`response.json()`	Converte o pacote de texto recebido em objeto JS.

Parte Final — Mantendo a Organização

Atividade prática 4 — Sincronizando seu Trabalho

Um bom desenvolvedor mantém seu histórico organizado. Após testar o funcionamento do ViaCEP, execute os comandos abaixo no seu terminal para enviar as alterações ao GitHub:

```
git add .
git commit -m "Roteiro 08: Implementando busca de CEP com Fetch API"
git push origin main
```